esprit Ecole Supérieure Privée d'Ingénierie et de Technologies	**Examen** Semestre : 1 ☐ 2 ■

Module : **Techniques de Conteneurisation**
Enseignant(s) : **UP Système**
Classe(s) : 4Arctic, 4Iosys & 4Sleam
Documents autorisés : OUI ☐ NON ■ Nombre de pages : 8
Date : **19/03/2025** Heure : 10H15 Durée : 1h30

N° Carte : ……..... Nom et Prénom : ……	Classe : ……..

Exercice 1 : Questions de réflexion (4pts)

1. Quelle est la différence entre les instructions **CMD** et **ENTRYPOINT** dans un Dockerfile ? (0,5pt)

   - **CMD** définit la commande par défaut qui sera exécutée lorsqu'un conteneur est lancé. Elle peut être remplacée lors de l'exécution du conteneur.

   - **ENTRYPOINT**, quant à lui, définit une commande qui ne peut pas être remplacée lors du lancement du conteneur (sauf si l'option --entrypoint est utilisée).

2. Quels sont les rôles respectifs de **containerd** et **runc** dans l'exécution des conteneurs ? (1pt)

   - **containerd** : moteur d'exécution de conteneurs de haut niveau, gérant leur cycle de vie, notamment la création, l'exécution, l'arrêt et la suppression des conteneurs, ainsi que la gestion des images et des volumes.

   - **runc** : moteur d'exécution bas niveau conçu uniquement pour exécuter des conteneurs en utilisant les fonctionnalités du noyau Linux (cgroups, namespaces). Il suit la spécification OCI et est utilisé par containerd pour l'exécution effective des conteneurs.

3. Quelle fonctionnalité du noyau Linux est utilisé pour appliquer des restrictions au niveau de l'utilisation de la mémoire, et pourquoi est-elle essentielle dans un environnement de conteneurisation ? (0,5 pt)

La fonctionnalité utilisée est cgroups (Control Groups).

Elle est essentielle pour limiter et isoler l'utilisation des ressources mémoire, évitant qu'un conteneur consomme toute la mémoire du système.

4. Que signifie le mode **"host"** dans le réseau Docker et dans quels cas est-il utilisé ? (0,5 pt)

Le mode host permet aux conteneurs d'utiliser la même interface que l'hôte. Il supprime donc l'isolation réseau entre les conteneurs, qui seront par défaut accessibles depuis l'extérieur. De ce fait, le conteneur utilisera la même adresse IP que la machine hôte.
Il est utilisé, par exemple, pour les applications nécessitant des performances réseau optimales ou lorsque des services ont besoin d'un accès direct au réseau de l'hôte.

5. Selon le fichier « docker-compose.yml » ci-dessous, identifier le type de stockage utilisé pour chaque service : (0,75 pt)

```yaml
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html

  db:
    image: mysql:latest
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mydb
    volumes:
      - db_data:/var/lib/mysql

  redis:
    image: redis:latest
    volumes:
      - /data

volumes:
  db_data:
```

- Service web : Bind Mount
- Service db : Volume nommé
- Service redis : Volume anonyme

6. Selon la capture ci-dessous, quels sont les rôles des nœuds **node1, node2 et node3** au sein du cluster Docker Swarm, Expliquer la différence. (0,75 pt)

```
[node1] (local) root@192.168.0.23 ~
$ docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
red3kzt2l11x8pookmb7kdabu *   node1      Ready     Active         Leader           27.3.1
vsx20xvsoi236ujg53enzjrxg     node2      Ready     Active         Reachable        27.3.1
3owtp6i72kuq3jai7v0hl5d3n     node3      Ready     Active                          27.3.1
```

- **node1 (Leader)** : Nœud manager principal qui prend des décisions clés concernant l'ordonnancement des tâches, la gestion de l'état du cluster et la

réplication des services. Il coordonne également avec les autres managers et workers.

- **node2 (Reachable Manager)** : Manager secondaire qui ne dirige pas mais peut prendre des décisions si le leader est indisponible, assurant ainsi la haute disponibilité du cluster.
- **node3 (Worker)** : Nœud qui exécute uniquement les conteneurs et les tâches assignées par les managers.

Exercice2 : (6 pts)

Dans le cadre de la préparation d'un environnement de test pour le site web d'une entreprise, la conteneurisation de l'application permet de simplifier son déploiement et sa gestion. La première tâche consiste donc à créer une image Docker basée sur Nginx afin d'héberger le site web de manière efficace.

1. Compléter le Dockerfile ci-dessous en ajoutant les instructions manquantes. (1,75 pt)

0.25pt pour chaque instruction

 # Utilisation de l'image de base Nginx

[….FROM….] nginx:latest

 # Définition du répertoire de travail dans le conteneur

[…WORKDIR….] /usr/share/nginx/html

 # Copie du fichier index.html dans le répertoire de Nginx

[…COPY….] index.html /usr/share/nginx/html/

 # Ajout et extraction automatique d'une archive contenant des fichiers statiques

[….ADD….] assets.tar.gz /usr/share/nginx/html/assets/

 # Exposition du port 80 pour permettre l'accès au serveur web

[…EXPOSE….] 80

 # Définition du point d'entrée pour exécuter une commande via le shell

[…ENTRYPOINT….] ["/bin/sh", "-c"]

 # Commande par défaut à exécuter, pouvant être remplacée lors du lancement du conteneur.

[…CMD….] ["nginx -g 'daemon off;'"]

2. Donner la commande pour créer une image Docker nommée **web-app** à partir de ce Dockerfile. (0,75 pt)

   docker build -t web-app .
   0,25PT 0,25pt 0,25pt

3. Ce Dockerfile est-il optimisé ? Si oui, mentionner une technique d'optimisation utilisée. Sinon, proposer des améliorations possibles pour optimiser l'image Docker. (0,5 pt) 0,25pt 0,25pt
   Le Dockerfile proposé n'est pas entièrement optimisé. Parmi les améliorations possibles, on peut citer l'utilisation d'un multi-stage build ou le remplacement de nginx:latest par nginx:alpine pour obtenir une image plus légère.

4. Trois instances ont été lancées à partir de cette image, et elles communiquent entre elles sans configuration réseau supplémentaire.
   a. Expliquer comment Docker gère cette communication entre conteneurs. (0,25pt)
      Docker gère la communication entre conteneurs via un réseau virtuel interne par défaut appelé "bridge".

   b. Quelle est l'interface réseau utilisée dans la question précédente et quel est son rôle ? (0,25 pt)
      docker0 qui est un pont virtuel auquel tous les NIC (Network Interface Card) créés dans des conteneurs s'y connectent. L'adresse IP sera allouée dans un sous-réseau de docker0 et la **passerelle** est **docker0**.

   c. Si l'on préfère connecter ces conteneurs dans un réseau isolé nommé **"web-app"**, quelle commande Docker utilise-t-on pour créer ce réseau ? (0,25pt)
      docker network create web-app

5. Lors de la phase de mise à jour, un administrateur système corrige une erreur d'affichage dans le fichier **index.html** du conteneur et met à jour la version de l'image de base dans le Dockerfile. Après avoir reconstruit l'image et relancé le conteneur, il constate que les modifications apportées au fichier **index.html** sont perdues.

   a. Expliquer pourquoi les modifications apportées au fichier index.html n'ont pas été conservées. (0,25pt)
   Car les modifications ont été faites à l'intérieur du conteneur sans persistance (les conteneurs sont éphémères)

b. Proposer une solution pour conserver les modifications apportées aux fichiers dans le conteneur, même après sa suppression ou sa recréation. (0.25 pt)

   Utiliser un volume Docker ou un bind mount.

   c. Donner la commande Docker permettant de lancer un conteneur nommé **nginx_container**, en utilisant la solution proposée dans la question précédente, sachant que tous les fichiers et les données du conteneur se trouvent sous /usr/share/nginx/html/. (0,75 pt)

Il faut accepter l'une des deux réponses suivantes

- **Commande Docker avec un volume :**

docker run --name nginx_container -v nginx_data:/usr/share/nginx/html web-app
0,25pt 0,25pt 0,25pt

- **Commande Docker avec un bind mount :**

docker run --name nginx_container -v /chemin/local:/usr/share/nginx/html web-app
0,25pt 0,25pt 0,25pt

6. Le conteneur **nginx_container** (créé dans la question précédente) génère des fichiers statiques que tu souhaites ensuite traiter dans un autre conteneur d'analyse. Ce conteneur d'analyse doit pouvoir accéder aux fichiers stockés par **nginx_container** pour accomplir ses tâches.
   a. Comment le conteneur d'analyse peut-il accéder aux données générées par le conteneur **nginx_container** ? (0,25pt)

   En utilisant un volume partagé entre les deux conteneurs.

   b. Donner la commande permettant de lancer le conteneur d'analyse **analysis-container** à partir de l'image **my-analysis-image**, tout en lui permettant d'accéder aux données générées par **nginx_container**. (0,75pt)

   docker run --name analysis-container --volumes-from nginx_container my-analysis-image
   0,25pt 0,25pt 0,25pt

Exercice3 : Problème (10pts)

Une application e-commerce est en cours de développement, intégrant :

- Un service Web pour l'affichage basé sur Nginx
- Une base de données MySQL pour le stockage des informations
- Un service API pour la gestion de la logique métier via une image personnalisée

Partie A

Lors de la conteneurisation de l'application, **l'équipe rencontre des problèmes liés aux configurations manuelles des conteneurs.**

1. Quel est le besoin pour résoudre ce problème, et quelle solution pourrait y répondre ? (0,5 pt)

   Le besoin est d'automatiser la configuration des conteneurs afin de réduire les erreurs humaines (0,25pt), La solution consiste à utiliser Docker Compose (0,25pt).
2. Le fichier suivant est fourni pour déployer une application multiconteneur basée sur deux services, web et db. Cependant, ce fichier comporte plusieurs erreurs :

```
version: '3.8'
Services:
  web:
    image : nginx:latest
    ports:
    80:80
  db:
    image: mysql:5.7
    environment :
      MYSQL_ROOT_PASSWORD: rootpassword
```

Figure 1

Identifier et corriger les erreurs présentes dans ce fichier. (0,5 pt)

- Services doit être en minuscules : **services**
- Le port mapping doit être sous forme de chaîne entre guillemets et précédé d'un tiret: **- "80:80"**
- Suppression des espaces avant :
  Image : → **image:**
  environment : → **environment:**

3. L'application inclut un service API qui utilise une image personnalisée construite à partir d'un Dockerfile.

Compléter le fichier de déploiement (figure 1) en y ajoutant le service API selon les spécifications suivantes : (1,75 pts)

- Construire l'image depuis le contexte **./api**
- Exposer le port 5000

Il faut accepter l'une des deux réponses suivantes:

Réponse 1 :

```
api:        (0,25 pt )
  build: (0,25 pt )
    context: ./api (0,5 pt )
    dockerfile: Dockerfile (0,25 pt )
  ports: (0,25 pt )
    - "5000:5000" (0,25 pt )
```

Réponse2:

```
api: (0,25 pt )
  build: ./api (0,75 pt )
  ports: (0,25 pt )
    - "5000:5000" (0,5 pt )
```

4. Pour préparer le déploiement en production, il est nécessaire d'améliorer le fichier de la **figure 1** en y apportant les modifications suivantes : (2 pts)
    - Ajouter un réseau nommé **mynetwork** adapté à un déploiement multi-hôtes.
    - Ajouter un volume nommé **myvolume**, avec le point de montage vers le conteneur défini sur le répertoire **/usr/share/nginx/html**.
    - Attribuer un nom explicite à chaque conteneur pour une meilleure identification.
    - Définir une politique de redémarrage pour relancer automatiquement un conteneur en cas d'échec.

Complétez le fichier ci-dessous, sachant que **la configuration est limitée au service web.**

L'image et le port sont déjà spécifiés dans le fichier de la figure 1, comme indiqué à la question 2, c'est pourquoi ils ne seront pas notés.

```
version: '3.8'
services:
  web:
      image: nginx:latest
      container_name: my_nginx (0,25pt)
      restart: on-failure (0,25pt)
      networks: (0,25pt)
         - mynetwork (0,25pt)
      volumes: (0,25pt)
         - myvolume:/usr/share/nginx/html (0,5pt)
     ports:
        - "80:80"

networks:
  mynetwork:
      driver: overlay (0,25pt)
volumes:
  myvolume:
```

Partie B

Une fois l'environnement de développement validé, il est nécessaire de déployer l'application e-commerce en production.

1. Proposer une solution pour déployer l'application en production en justifiant votre choix. (0,5 pt)
   Pour déployer l'application en production, **Docker Swarm** est une solution idéale. Il permet d'orchestrer plusieurs conteneurs sur différents hôtes, assurant scalabilité, haute disponibilité et reprise automatique en cas de panne.

2. Pour cela, on va mettre en place un cluster composé de 4 nœuds comme suit :

- **Nœud 1 (Leader)** : Adresse IP 192.168.40.100
- **Nœud 2 (Manager)** : Adresse IP 192.168.40.101
- **Nœud 3 (Worker1)** : Adresse IP 192.168.40.102
- **Nœud 4 (Worker2)** : Adresse IP 192.168.40.103

   Indiquer la commande permettant d'initialiser le cluster sur le nœud leader. (0,5 pt)
   docker swarm init --advertise-addr 192.168.40.100
   0,25pt 0,25pt

3. Citer les commandes permettant de générer les tokens :

- Pour le nœud manager : (0,5 pt)
  docker swarm join-token manager
- Pour le nœud worker : (0,5 pt)
  docker swarm join-token worker

4. Une fois le cluster configuré, l'objectif est de répartir toutes les tâches sur les nœuds managers et workers, **à l'exception du nœud leader.**
   Comment empêcher le nœud leader de recevoir des tâches ? Spécifier la commande. (0,5 pt)

   docker node update --availability drain Nœud 1
   0,25pt 0,25pt

5. Avant de déployer l'application complète, un test du cluster sera effectué avec le service web.
   Indiquer la commande permettant de lancer un service nommé "web", basé sur l'image **nginx:latest**, avec 3 réplicas et exposant le port 80. (1,25 pt)

   docker service create --name web --replicas 3 -p 80:80 nginx:latest
   0,25pt 0,25pt 0,25pt 0,25pt 0,25pt

6. Citer la commande permettant de vérifier l'état du service et d'afficher la liste des tâches en cours d'exécution pour ce service. (0,25 pt)

   docker service ps web

7. Après la validation du test, citer la commande permettant de déployer l'application complète sur le cluster en utilisant le fichier de la figure1. (0,5 pt)
   docker stack deploy -c docker-compose.yml myapp
   0,25pt 0,25pt
8. Indiquer la commande permettant au nœud Worker 2 de quitter le cluster. Que se passe-t-il pour les tâches en cours d'exécution sur ce nœud au moment de son départ ? (0,75 pt)

   docker swarm leave –force 0,25pt

   Les tâches en cours seront redistribuées sur les autres nœuds. 0,5pt